VU AI HUB · LECTURE NOTES

Data Structures and Algorithms for AI

Year 2, September to October 2026

Written by Inaaya and shared on Notion. Original page: cheddar-wall-f3b.notion.site/3c09801a9309801abef5d205e676cb77

Contents

1. 31st of August
2. 7th of September
3. 7th of September (WG)
4. 14th of September
5. 14th of September (WG)
6. 21st of September
7. 21st of September (WG)
8. 28th of September
9. 28th of September (WG)
10. 5th of October
11. 5th of October (WG)
12. 12th of October
13. 12th of October (WG)

31st of August

An algorithm is a computational procedure which takes an input and, through a sequence of steps, transforms it into an output

An instance is one specific filled-in example of the problem, actual numbers and an actual graph. To be a real instance, it must have all the data you need and follow the rules

A correct algorithm must work on every single instance, it shouldn't halt (get stuck in an infinite loop) or halt wrong (stops, but gives the wrong answer), it should halt on the right answer

4 types of big O

O(1) - Constant

- The operation takes the same amount of time no matter how much data there is
  - Grabbing the first item off a grocery list → you never read the rest of it (no matter the amount of products on there)

O(n) - Linear

- As the data grows, the time grows at the same rate (double the data → double the work)
  - Searching a list of names one by one → you don't know where it is (as the case of the previous example), so worst case you check them all
  - It's longer than the constant, but still very predictable

O(n^2) - Quadratic

- Every item has to interact with every other item, so the work explodes as data is added
  - Multiplying the data size by itself
  - A group where every person has to shake hand with every other person

O(log n) - Logarithmic

- Each step throws away half of what's left → doubling the data adds only one more step
  - Splitting a book in half, then half again, to find a page

Data structures

The Array

- Each item has a fixed position
  - Cannot just squeeze an additional item → because the array has a fixed size
- Items sit side by side in memory
  - contiguous sequence of bytes
  - the index is an address
- You can quickly go to any item by knowing their index (address)

Concrete numbers

a = start

b = size of one item

`address = a + b x (index - s)`

Operations

Retrieving/replacing an item is Constant (O(1)), because it only takes one step

If you want to add an item to an array, you can't just squeeze it in, you can however copy your original array and then make a new one while adding your new item simultaneously (wherever you want, not just add the end or beginning) (O(n))

The same goes for deleting an item in an array, you can't just do that and leave an empty cell, you need to do a calculation where every item after the empty cell moves 1 place back (O(n))

When to use

- You access by position
- You read far more often than you resize
- All elements are the same size
- You want the smallest possible memory overhead

When not to use

- The data grows and shrinks constantly
- You insert or delete → each one is O(n)
- You search by value rather than index

The Matrix

You need two indices to get one element → the row and column

Memory however is one-dimensional → we need to decide the order in which the twelve values are laid out → and that decision has consequences

Row-major order

Take the whole row, then the whole second row, and so on …

1 2 2 3

3 3 4 6 → 1 2 2 3 3 3 4 6 2 8 9 5

2 8 9 5

`index = n(i - 1) + j`

i - 1 allows you to skip whole rows (each holding n elements) to get on the row you need to be to find your number. When you're on the right row you look at j which gives you the position your number is on.

Column-major order

Take the whole first column, then the whole second column, and so on …

1 2 2 3

3 3 4 6 → 1 3 2 2 3 8 2 4 9 3 6 5

2 8 9 5

`index = i + m(j - 1)`

j - 1 allows you to skip complete columns (each holding m elements) to get to the column you need to be to find your number. When you're on the right column you look at i which gives you the position your number is on.

Operations

Retrieving an item is Constant (O(1)) because it only requires one index calculation which counts as one step

When you want to search for an item there is no order you need to follow, you can do that in any way you want, it will take the amount of steps you get when you multiply the amount of rows with the amount of columns (O(m*n)), this is the same as inserting a row

Traversing a row (O(n)) or traversing a column (O(m)) depend on the amount of rows and columns

When to use

- Images, grids, tables, adjacency matrices
- Two indices identify one value
- Rows all have the same length

When not to use

- Sparse data → you store mostly zeros
- Ragged rows of different lengths
- Rows or columns are inserted often

The Stack

Last in → first out

Worst case scenario, your item is at the bottom of the stack, because you need to get every item on top of it out first (O(n))

Inserting an item can only be at the top (Pushing) (O(1))

Deleting an item can also only go from the top (Pop) (O(1))

With the stack you need to take overflow and underflow into consideration.

- Overflow - S.top would exceed S.size
  - It has a limit, pushing onto a full stack should be guarded
- Underflow - S.top is already 0
  - You can't go into negatives, popping an empty stack should be guarded

The three procedures

STACK-EMPTY (S)

```
if S.top == 0
   return TRUE
else return FALSE
```

PUSH (S, x)

```
if S.top == S.size
   error “overflow”
else S.top = S.top + 1
   S[S.top] = x
```

POP (S)

```
if STACK-EMPTY (S)
   error “underflow”
else S.top = S.top - 1
   return S[S.top + 1]
```

When to use

- Undo/redo history
- Backtracking and depth-first search
- Reversing a sequence

When not to use

- You need an item in the middle
- You need to search by value
- Fairness or arrival order matters

The Queue

First in → First out

Worst case scenario, your item was stored last, because you can only retrieve from the start (O(n))

Unlike arrays you can only add (enqueue) at the end and retrieve (dequeue) only from the start (O(1))

The Q tail is one step after the item on the end, the Q head is at the number of the start

ENQUEUE (Q, x)

```
Q[Q.tail] = x
if Q.tail == Q.size
   Q.tail = 1
else Q.tail = Q.tail + 1
```

DEQUEUE (Q)

```
x = Q[Q.head]
if Q.head == Q.size
   Q.head = 1
else Q.head = Q.head + 1
return x
```

When to use

- Task schedulers
- Breadth-first search
- Streaming → you never need to look back

When not to use

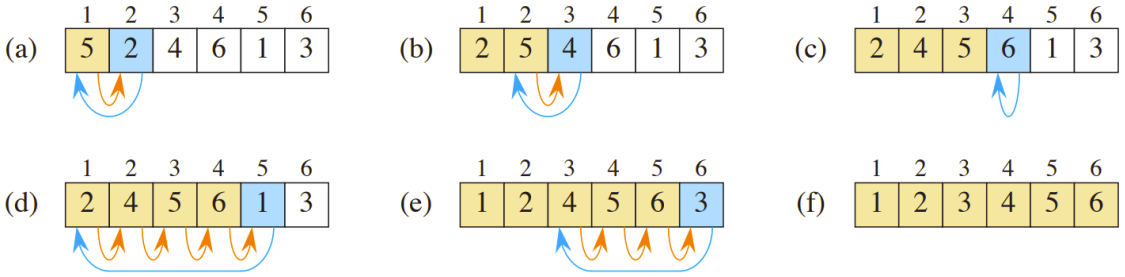
- Some items must jump the line by priority
- You need random access or search by value
- You must revisit what has already been

7th of September

Sorting Algorithms

Insertion Sort

- Given an array of un-sorted elements, begin by comparing the second element to the first (a), the third to the first two (b), etc.
  - Like sorting a hand of playing cards: you hold cards 1 to i-1 already sorted in your hand, then pick up a card i (call it the key) and slide it backward into its correct spot among the cards you're already holding



Loop invariants

- To prove a statement is a loop invariant, we must show:
  - it holds true prior to the first iteration of the loop (Initialization)
    - let us show A[1 : i − 1] is sorted before the first loop iteration
      - Take i=2, A[1 : i − 1]= A[1]
      - An array with a single element is sorted without question
      - The loop invariant thus holds before the first loop iteration
  - if it holds true before an iteration, it remains true before the next iteration (Maintenance)
    - let us show A[1 : i] is sorted after a loop iteration, if A[1 : i − 1] was sorted before the iteration
      - Notice the iteration splits A[1 : i] into three subarrays: A[1 : j], A[j + 1], A[j+2 : i]
        - A[1 : j] remains untouched, thus sorted by hypothesis
        - A[j + 1] is undeniably sorted
        - A[j+2 : i] holds the values, shifted by one, of A[j+1 : i−1], so it is sorted by hypothesis
      - Where j is such that and A[j] < key = A[j+1] < A[j+2]
      - Each subarray being sorted and having, A[j]<A[j+1 <A[j+2], we conclude the subarray A[1:i] is sorted after the iteration
  - upon termination of the loop, the invariant (along with the condition which caused the loop to terminate) help to determine the algorithm is correct (Termination)
    - let us show the algorithm is correct, using the loop invariant, A[1 : i − 1] is sorted before iteration i
      - Assume it's already sorted
      - When the loop terminates, i=n+1, thus A[1 : i − 1] = A[1 : n] is sorted
      - This immediately concludes that the algorithm is correct

```
SUM-ARRAY(A, n)
1  sum = 0
2  for i = 1 to n
3      sum = sum + A[i]
4  return sum
```

Initialization: Show initially sum = A[0 : 0]

Maintenance: Show after one iteration sum contains sum of A[0 : i] if beforehand has sum A[0 : -1]

Termination: Prove I end with sum of A[0 : n-1]

Running Time Analysis

- Denote by ck the cost of running line k
- Let ti be the number of times the while loop is run, for given i

INSERTION-SORT(A, n)		*cost*	*times*
1	**for** $i = 2$ **to** n	c_1	n
2	$key = A[i]$	c_2	$n-1$
3	// Insert $A[i]$ into the sorted subarray $A[1 : i-1]$.	0	$n-1$
4	$j = i-1$	c_4	$n-1$
5	**while** $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^{n} t_i$
6	$A[j+1] = A[j]$	c_6	$\sum_{i=2}^{n}(t_i - 1)$
7	$j = j-1$	c_7	$\sum_{i=2}^{n}(t_i - 1)$
8	$A[j+1] = key$	c_8	$n-1$

- In the best case scenario (fastest), the array is initially sorted and the while loop exits immediately, so ti = 1 for any i
- In the worst case scenario (slowest), the array is in reverse order, so the key must be compared with each element of the sorted subarray, A[1 : i-1], thus ti = size of sorted subarray= i
- In an 'average case', half the elements of the sorted subarray are langer than the key, so ti ~ i/2, roughly as bad as the worst case scenario

Worst case scenario analysis

- Let T(n) be the total running time in the worst case scenario (ti = i).

$$T(n) = c_1 n + (c_2 + c_4 + c_8)(n-1) + c_5 \sum_{i=2}^{n} t_i + (c_6 + c_7) \sum_{i=2}^{n} (t_i - 1)$$

$$= c_1 n + (c_2 + c_4 + c_8)(n-1) + c_5 \sum_{i=2}^{n} i + (c_6 + c_7) \sum_{i=2}^{n} (i - 1)$$

Change all the ti's to just i's because ti = i

$$T(n) = c_1 n + (c_2 + c_4 + c_8)(n-1) + c_5 \sum_{i=2}^{n} i + (c_6 + c_7) \sum_{i=2}^{n} (i - 1)$$

$$= c_1 + (c_1 + c_2 + c_4 + c_8)(n-1) + (c_5 + c_6 + c_7) \sum_{i=2}^{n} i + (c_6 + c_7) \sum_{i=2}^{n} (-1)$$

$$= c_1 + (c_1 + c_2 + c_4 + c_8 - c_6 - c_7)(n-1) + (c_5 + c_6 + c_7) \sum_{i=2}^{n} i$$

$$= \alpha + \beta(n-1) + \theta \sum_{i=2}^{n} i$$

substituting, $\alpha = c_1$

$$= \alpha + \beta(n-1) + \theta \left(\frac{n(n-1)}{2} - 1 \right)$$

$\beta = (c_1 + c_2 + c_4 + c_8 - c_6 - c_7)$

$\theta = c_5 + c_6 + c_7$

Order of Growth

$$T(n) = \alpha + \beta(n-1) + \theta\left(\frac{n(n-1)}{2} - 1\right)$$
$$= \frac{\theta}{2}n^2 + \frac{2\beta - \theta}{2}n + (\alpha - \beta - \theta)$$

- To further simplify our time running analysis, we focus only on the order of growth, the factor from the leading term: n^2
- We say insertion sort has worst case running time of Θ(n^2)
- This running time analysis is particularly relevant for large enough inputs

---

Θ notation and Big-O notation

- Big-O notation gives an upper bound, it grows no faster than this
- Ω notation gives a lower bound, it grows no slower than this
- Θ notation for if a function is both, it grows exactly at this rate, say O(n^3) and Ω(n^3), then it is Θ(n^3)

In practice, Big-O notation is used interchangeably with Θ-notation

---

Algorithm design

Incremental algorithms

- Grow the solved part by one piece at a time, insertion sort is incremental, the sorted region just grows by one card each round
- At each step, it increases the size of the currently sorted subarray A[1:i-1] by one, by appropriately placing the element A[i]
- Notice the algorithm always makes use of the same action: placing one element in its appropriate place within a sorted list

Divide and conquer algorithms

- Another useful algorithm design involves a recursive structure: to solve a problem, the algorithm calls itself (recurses) to solve related subproblems
- These algorithms typically divide and conquer: breaking the problem into several smaller subproblems, then combining

The divide and conquer method relies on three steps:

- Divide the problem into one or more subproblems which are smaller instances of the original problem.
- Conquer the subproblems by solving them recursively
- Combine the subproblem solution to form a solution to the original problem

---

Merge sort

One good example is merge sort which:

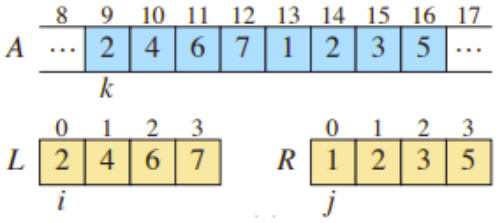
- splits an array into two subarrays down the middle,
- recurses down to smaller and smaller arrays,

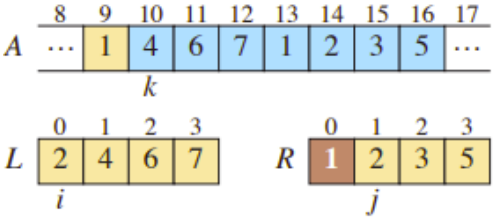
- and finally combines the adjacent sorted subarrays, into one singular sorted array.

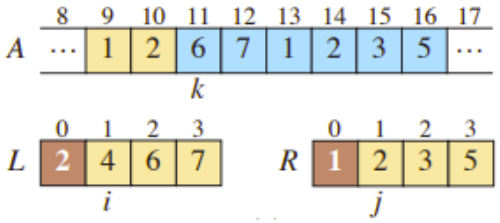
```
MERGE-SORT(A, p, r)
1  if p ≥ r                          // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                   // midpoint of A[p : r]
4  MERGE-SORT(A, p, q)               // recursively sort A[p : q]
5  MERGE-SORT(A, q + 1, r)           // recursively sort A[q + 1 : r]
6  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7  MERGE(A, p, q, r)
```

Merge procedure

- Analogy: you have two-already sorted piles of cards face-up. Look at just the top card of each pile, whichever is smaller, play it and repeat. Once one pile runs out, dump the rest of the other pile on top (it's already sorted, so no more comparing needed). This step alone costs Θ(n), every card gets looked at and placed exactly once.









Pseudocode

```
MERGE(A, p, q, r)
 1  n_L = q − p + 1          // length of A[p : q]
 2  n_R = r − q              // length of A[q + 1 : r]
 3  let L[0 : n_L − 1] and R[0 : n_R − 1] be new arrays
 4  for i = 0 to n_L − 1     // copy A[p : q] into L[0 : n_L − 1]
 5      L[i] = A[p + i]
 6  for j = 0 to n_R − 1     // copy A[q + 1 : r] into R[0 : n_R − 1]
 7      R[j] = A[q + j + 1]
 8  i = 0                    // i indexes the smallest remaining element in L
 9  j = 0                    // j indexes the smallest remaining element in R
10  k = p                    // k indexes the location in A to fill
11  // As long as each of the arrays L and R contains an unmerged element,
    //      copy the smallest unmerged element back into A[p : r].
12  while i < n_L and j < n_R
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
18      k = k + 1
19  // Having gone through one of L and R entirely, copy the
    //      remainder of the other to the end of A[p : r].
20  while i < n_L
21      A[k] = L[i]
22      i = i + 1
23      k = k + 1
24  while j < n_R
25      A[k] = R[j]
26      j = j + 1
27      k = k + 1
```

Analysis of merge sort

We compute the worst case running time of merge sort, T(n), recursively, distinguishing the three stages of the algorithm:

- Divide computing the middle of the array takes constant time Θ(1)
- Conquer recursively solving two subproblems, each of size n/2, takes a total running time of 2T(n/2)
- Combine The merge procedure takes Θ(n) time

Adding them together, we get

T(n) = 2T(n/2) + Θ(n) = Θ(nlog(n))

The worst case running time of merge sort, T(n) is:

T(n) = 2T(n/2) + Θ(n) = Θ(nlog(n))

To understand this equality, think "How many times did we recursively call the algorithm?"

- At each step, we make the subproblems half the size. So, at step i, the subproblems are each of size n/(2i)

- We stop this, when the subproblems are size 1, so will split the problem into subproblems log2(n) times
- Solving the subproblems, and merging to double in size the sorted subarrays, will take Θ(n) each time

Hence the algorithm takes T(n) = Θ(nlog(n))

7th of September (WG)

Bubble Sort

Start from the back and compare the last one with the one that comes before, keep doing that with everything that comes after it, when you're done with the first number you look at the back again and do it all over again until everything is in the right spot

Iterative vs. Recursive Approach

Iteration

- Uses loops (for, while)
- Tracks progress using variables
- Usually uses less memory

Recursion

- Function calls itself (on smaller problem)
- Passes information through its arguments
- Requires a base case (i.e. stopping criteria)
- Often elegant and expressive

14th of September

Linked lists

- Array: order = index position, linked list: order = pointers between objects
- Doubly linked list node has 3 things: `key`, `prev`, `next`
- Search: start at `L.head`, walk via `.next` until you find the key or hit NIL

  To find the first element with key k in the list L, run:

```
LIST-SEARCH (L, k)
    x = L.head
    while x ≠ NIL and x.key ≠ k
        x = x.next
    return x
```

  To add an element, x, to the front of the list L, run:

```
LIST-PREPENT (L, x)
    x.next  = L.head
    x.prev = NIL
    if L.head ≠ NIL
        L.head.prev = x
```

```
    L.head = x
```

To add an element, x, following an element, y, in the list, run:

```
LIST-INSERT (x, y)
    x.next  = y.next
    x.prev = y
    if  y.next  ≠ NIL
        y.next.prev = x
    y.next  = x
```

Sentinel trick: add one fake 'dummy' node `L.nil` that stands in for NIL and links the list into a circle (lase node's `next` points back to it, first node's `prev` points to it too)

- head = `L.nil.next`, `tail` = `L.nil.prev`
- It kills several edge cases (empty list? list of one? inserting at the front?), one piece of code handles everything
- You can temporarily stuff the key you're searching for into `L.nil.key`, then the search loop never has to check 'did I fall off the list' (`x ≠ NIL`), only 'did I find it'

```
LIST-INSERT' (x, y)
    x.next  = y.next
    x.prev = y
    y.next.prev = x
    y.next  = x
```

Deleting an element is as easy as splicing:

```
LIST-DELETE (x)
    x.prev.next  = x.next
    x.next.prev = x.prev
```

Stack using a linked list

- Push = prepend a new node at the head
- Pop = read the head's key, then delete the head

Dictionaries & Hash Tables

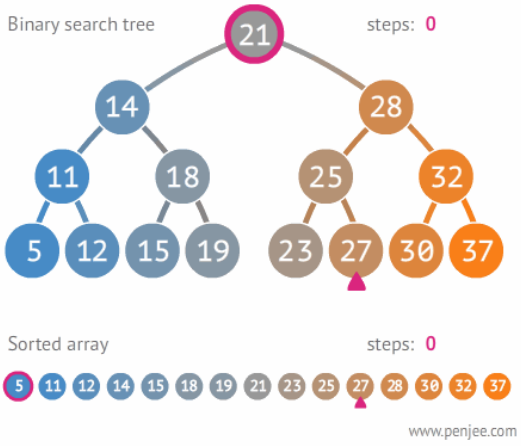
- A dictionary structure only needs Insert/Search/Delete (no min, max, successor)
  - Dumbest dictionary: direct-address table, slot `k` in an array holds the element with key `k`. O(1) but wastes massive memory if the universe of possible keys is way bigger than the keys you actually use
- A hash table is the fix for that, a hash function `h` squeezes the big universe of keys down in to a small table of size `m`: `h: U → {0,...,m-1}`
- Division method: `h(k) = k mod m`, it's fast but works best when `m` is prime and not close to a power of 2
- Collision = two keys land on the same slot, fix = chaining: each slot points to a (doubly) linked list of everything that hashed there
- With chaining:
  - Insert = prepend into T[h(key)] → O(1)
  - Search = list-search inside that one chain → worst case = proportional to chain length
  - Delete = O(1) if the chains are doubly linked (you already have the pointers, no need to re-search)
  - Moral: a good hash function spreads keys evenly so chains stay short

Trees

- General tree (any number of children): give each node 3 pointers - `p` (parent), `left-child` (just the leftmost one), `right-sibling` (the next one over). That's enough to represent arbitrary branching without a variable-size array of children
- Binary tree node: `key`, `p`, `left`, `right`
- Print every key (tree walk), recursive:
  - `if x ≠ NIL: walk(x.left); print(x.key); walk(x.right)` - this prints in sorted order for a BST
- Non-recursive version (exercise): use your own stack instead of relying on the recursion's call stack, push every node as you go left, then pop+print, then go right

---

Binary Search Trees (BST)

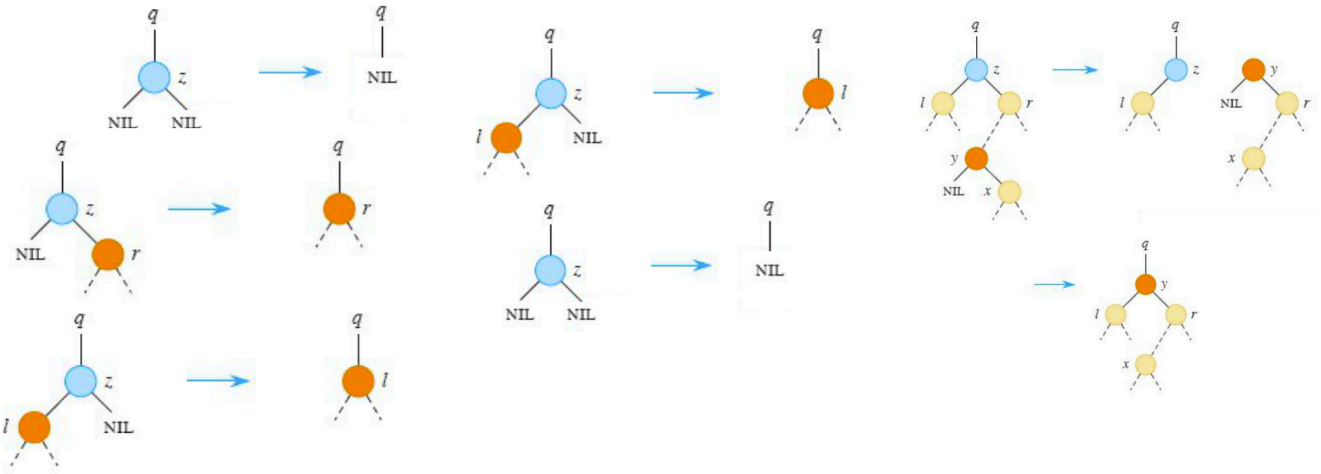


- For any node `x`, everything in its left subtree ≤ `x.key` ≤ everything in its right subtree
- Search: compare `k` to `x.key`, go left if smaller, right if bigger, stop at NIL (not found) or a match, the cost = tree height
- Min = keep going left until you can't, max = keep going right until you can't
- Insert: walk down exactly like search until you fall off the tree (hit NIL), attach the new node there as a leaf (track the parent as you go so you know where to hang it)

14th of September (WG)

Deleting in a BST tree - 4 cases

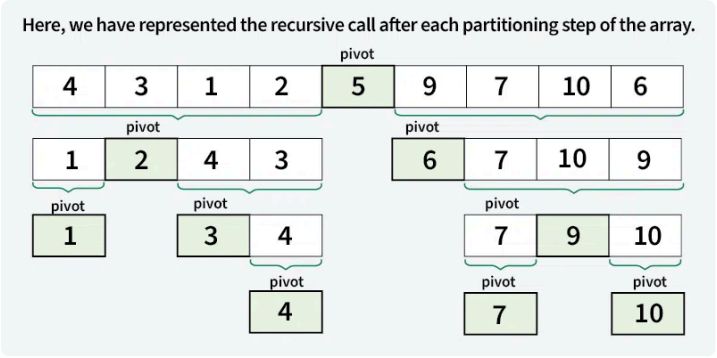
1. `z` has no left child → just put `z`'s right child in `z`'s spot (could be NIL)
2. `z` has no right child → symmetric: put `z`'s left child in `z`'s spot
3. `z` has both children, and `z`'s right child `y` has no left child → `y` slides straight up into `z`'s position (and keeps `z`'s left subtree as its own left subtree)
4. `z` has both children, and the successor `y` is further down (`y ≠ z.right`) → first yank `y` out where it currently sits (replace `y` with `y`'s own right child, since `y` can never have a left child), then drop `y` into `z`'s spot



- `y` (the successor) is always the right choice: successor = smallest key bigger than `z.key = TREE-MINIMUM(z.right)`. Being a 'minimum', it can never have a left child, that's exactly why it can slot in so cleanly
- `TRANSPLANT(T, u, v)` is the helper that does the actual 'swap subtree `v` into the spot where `u` used to be', just fixes up whichever parent pointer (`T.root`, or `parent.left`, or `parent.right`) used to point at `u`.

21st of September

Quick Sort



- Divide: partition into two subarrays, such that all elements in one subarray are smaller than a chosen pivot, and those in the other subarray are in larger than this pivot
- Conquer: recursively calls itself to sort the smaller arrays
- Combine: join the two arrays, the union is immediately sorted, there is no need for any more steps

```
QUICKSORT(A, p, r)
    if p < r
        q = PARTITION(A, p, r)
        QUICKSORT(A, p, q-1)
        QUICKSORT(A, q+1, r)
PARTITION(A, p, r)
    x = A[r] // pivot
    i = p - 1 // boundary of "small" region
    for j = p to r-1
```

```
    if A[j] ≤ x
      i = i + 1
      exchange A[i] with A[j]
  exchange A[i+1] with A[r] // put pivot in its final spot
  return i + 1
```

- Analogy: `i` is a wall, everything left of the wall is ≤ pivot, `j` scans forward, whenever it finds something ≤ pivot, the wall moves up one and that element gets swapped behind the wall

Partition's loop invariant

At the start of every iteration of the for-loop, three things are true:

- Everything from `p` to `i` ≤ pivot
- Everything from `i+1` to `j-1` > pivot
- `A[r]` = pivot (untouched so far)

Two cases each iteration:

- `A[j] > x` → just increment `j`, nothing else changes (the '>' region grows)
- `A[j] ≤ x` → increment `i`, swap `A[i]` and `A[j]`, then increment `j` (the '≤' region grows by one)

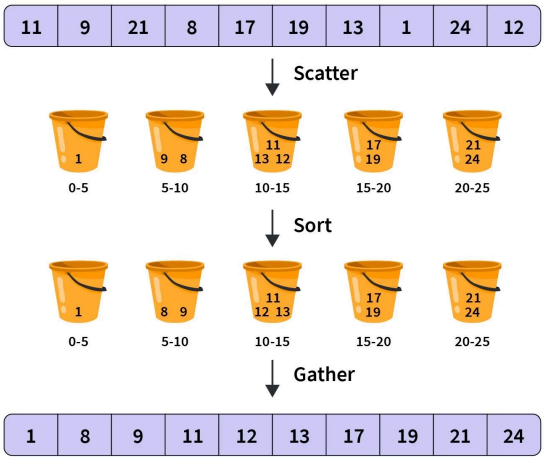
Running time of PARTITION: O(n), single pass, constant work per element

Running time

- Worst case: pivot always splits n-1 / 0 (the split is not even, everything is on one side and nothing is on the other), this happens when the array is already sorted in descending order
- Best case: pivot always splits evenly in half
- Average case: for random input, expected running time is O(n log n), for example if the first split is like the worst case one and the second one is like the best case one you don't really use much
- Hoare's partition: uses two pointers `i` (scans up from left) and `j` (scans down from right), swapping only when both are mis-sorted relative to pivot

---

Bucket Sort

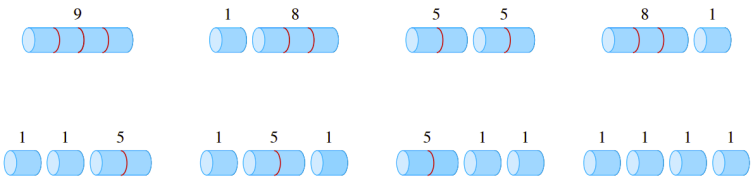


- Assumption: input values are uniformly distributes in [0, 1)
- Idea: like a hash table, make `n` buckets, drop each element `A[i]` into bucket `⌊n·A[i]⌋` (this spreads values evenly across buckets). Sort each small bucket with Insertion Sort, concatenate buckets in order
- Running time: O(n), linear, because with uniform data each bucket gets ~1 element on average

---

Rod Cutting Problem

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30



- Setup: a rod of length `n`, price table `p_i` for each possible cut length `i`, the goal is to maximize the revenue from cutting the rod into pieces and selling them
- Recurrence: `r_n = max{ p_i + r_{n-i} : 1 ≤ i ≤ n }` - try every possible first cut, add its price to the best answer for the remaining piece
- Naive recursive version: (`CUT-ROD`) re-solves the same sub-length over and over, exponential time
- Bottom-up dynamic programming fix (`BOTTOM-UP-CUT-ROD`): build an array `r[0 … n]` from the smallest length up, so every subproblem is solved exactly once
- Reconstructing the actual cuts: add a second array `s[j]` that remembers which first cut `i` gave the best value for length `j`. Then `PRINT-CUT-ROD-SOLUTION` walks it like a linked list: print `s[n]`, subtract it from `n`, repeat until `n=0`

Optimal Binary Search Trees

Difference with a normal BST is that it wants to minimize the average search time, given that some keys get searched way more often than others. That might mean putting a very frequently-searched key near the root, even if that makes the tree lopsided and 'unbalanced' by normal standards, it consists of:

- Keys `k_1, …, k_n`: the actual things people search for, each with a probability `p_i` of being searched (how popular is the key)
- Dummy keys `d_0, …, d_n`: these represent 'you searched for something that isn't in the tree', each has probability `q_i`, they always end up as leaves
- All the `p`'s and `q`'s together add up to 1

Cost of a search = how many steps down the tree you take:

> **Expected cost = for every key, (its depth in the tree + 1) × (how often people search for it) — summed over all keys and dummy keys**

- Popular key sitting deep in the tree is not wanted because it costs a lot, it's better when its near the root

Dynamic Programming

- If the whole tree is optimal, then whatever its left subtree and right subtree are, they must also individually be the optimal trees for their own smaller sets of keys
- So you try every key as the root, and for each choice, assume you already know the best possible left and right subtrees, pick whichever root gives the lowest total cost
- `e[i, j]` = is the cost of the best possible tree built from keys `k_i` through `k_j`
- If you try `k_r` as the root of that range:

```
e[i, j] = e[i, r-1] (cost of best left subtree)
    + e[r+1, j] (cost of best right subtree)
    + w[i, j] (a flat "tax" for adding one more level)
```

- `+ w[i, j]` means that every key/dummy in this range got pushed one level deeper by adding a new root above them. Since cost = depth x probability, each one's cost went up by exactly it's own probability
  - So the tax = sum of all the probabilities in that range
- The algorithm just tries every possible `r` for every possible range `[i, j]`, and keeps whichever root gives the smallest `e[i, j]`, store that best root in a `root[i, j]` table

21st of September (WG)

0/1 Knapsack

- Items have value `v_i` and size `s_i`, the bag holds capacity `C`. You pick a subset maximizing total value without exceeding `C`. Each item is taken whole or not at all (0/1)
- for item `i` and remaining capacity `C`, you only have two choices, take it or skip it:

  `MV(i, C) = max( MV(i-1, C), // skip item i`

  `MV(i-1, C-S_i) + V_i ) // take item i (only if S_i ≤ C)`
- Building the table: rows = items added one at a time, columns = capacity 0 … C. Each cell reuses two cells from the row above (skip-value and take-value). Fill left-to-right, top-to-bottom
- Reconstructing which items were taken (backtrack from the bottom-right cell):
  - Compare `DP(i, C)` to `DP(i-1, C)`
  - If equal → item i was not taken, move up a row, same `C`
  - If different → (`DP(i,C)` is bigger) → item i was taken, move up a row AND subtract that item's size from `C`
  - Repeat until you reach row 0

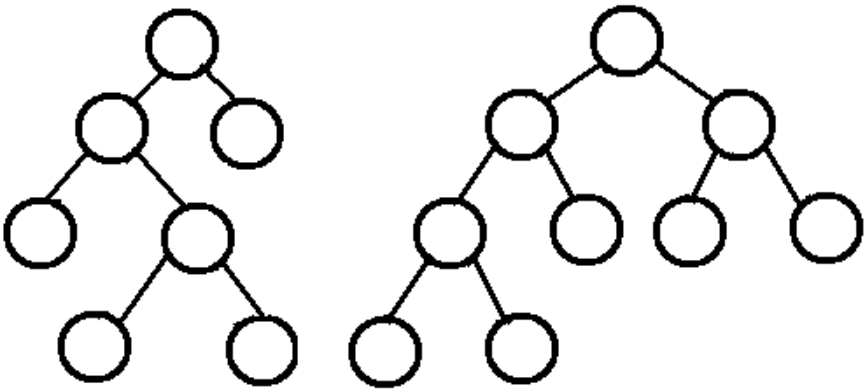
28th of September

Bucket Sort

- Same idea as before: spread `n` numbers into `n` buckets, sort each bucket, concatenate
- The actual formula to pick which bucket a number goes into
  - `normalized = num / (max + 1)` - squishes the number into the range [0,1)
  - `bucketNo = int(n * normalized)` - scales it up to pick a bucket index 0 to n-1
- If the items are clustered, most of them just pile into one or two bins makes you lose the O(n) guarantee and degrades toward O(n^2) in the worst case

---

Heaps



- The problem heaps solve: 'how do I always keep the biggest (or smallest) item within easy reach, without fully sorting everything
- A heap is a complete binary tree
  - Binary → every node has at most 2 children
  - Complete → every level is completely filled except possibly the last, and the last level fills left-to-right with no gaps
- Max-heap property: `A[Parent(i)] ≥ A[i]` for every node, every parent is ≥ its children. The consequence is that the biggest element is always at the root, and every subtree of a max-heap is itself a max-heap
- Min-heap: same idea but flipped: `A[Parent(i)] ≤ A[i]`, smallest element at the root
- When you're storing a heap you don't need any pointers, just a flat array (read the tree breadth-first, level by level, left to right in the array)

Max-Heapify

- Used when node `i`'s subtrees are already valid max-heaps, but `A[i]` itself might be smaller than one of its children (a single violation at the top)
- Fixed in 3 steps
  1. Find the largest among `A[i]]`, `A[left(i)]`, `A[right(i)]`
  2. If a child is bigger, swap it with `A[i]`
  3. Recurse into whichever child just received the old (now-too-small) value, because the problem may have 'sunk down' into that subtree
- Running time: O(h), where h = height of node `i`, you do at most one swap per level going down, worst case all the way to the leaf

Build Max-Heap

- `for i = ⌊n/2⌋ downto 1: MAX-HEAPIFY(A, i)` - only loop over nodes that have children (leaves are already trivially valid heaps, nothing to fix), and go from the bottom up
- Using bottom-up because by the time you call Max-Heapify on node `i`, both its children's subtrees are already valid heaps (you fixed them first), so Max-Heapify's assumption 'children are valid heaps' is always satisfied
- The loop invariant: before every iteration, all nodes `i+1, …, n` are each the root of a valid max-heap
  - Starts true trivially (leaves are heaps by default)
  - Stays true because each node's children (indices `2i` and `2i+1`, both > 1) were already processed and fixed

    - Ends with node 1 (the whole array) being a valid max-heap
- Running time: even though each Max-Heapify call can cost O(h), most nodes are near the bottom (where h is small) and only a few are near the top (where h is large), the sum works out to O(n)

Heapsort

```
HEAPSORT(A, n)
    BUILD-MAX-HEAP(A, n)
    for i = n downto 2
        exchange A[1] with A[i] // move the current max to the end
        A.heap-size = A.heap-size - 1 // shrink the heap (shut that slot out)
        MAX-HEAPIFY(A, 1) // fix the heap at the root
```

- You build the heap and then 'steal the root' repeatedly
- You still need the loop after building the heap once, a max-heap guarantees parent ≥ children, it says nothing about the order between siblings or different branches. So you can't just read the array off directly, you have to repeatedly extract the current max
- Each iteration you swap the max (root) to the back of the array (its final sorted position), shrink the heap by one, then Max-Heapify to restore the property at the top
- The loop invariant: at the start of each iteration, `A[1..i]` is a max-heap holding the `i` smallest remaining elements, and `A[i+1..n]` already holds the rest, correctly sorted
- Running time: O(n log n), build Max-Heap costs O(n), then n-1 extractions extractions each costing O(log n) for the Max-Heapify fix-up

---

Graph Theory

- A graph `G = (V, E)`: `V` = set of vertices (nodes), `E` = set of edges (connections between pairs of vertices)
- Order `|V|` = number of vertices
- Size `|E|` = number of edges
- `u ~ v` means u and v are adjacent (there's an edge between them)
- Neighborhood `N(v)` = all vertices adjacent to v
    - Closed neighborhood `N[v]` = `N(v)` plus v itself
- Degree `deg(v)` = number of edges touching v
    - `Δ(G)` = max degree in the graph
    - `δ(G)` = min degree in the graph
- Handshaking Lemma = sum of all degrees = `2*|E|` (every edge gets counted twice, once from each endpoint)
    - Consequence: the number of vertices with odd degree is always even

Traverse the graph

- Walk: vertices and edges can repeat, anything goes
- Trail: no repeating edges (but vertices can repeat)
- Path: no repeating vertices (which automatically means no repeating edges either, the strictest)
- Cycle: a path that loops back to where it started

Connectivity

- A graph is connected if every pair of vertices has a path between them, otherwise it's disconnected
- Each separate 'island' of connected vertices is called a connected component

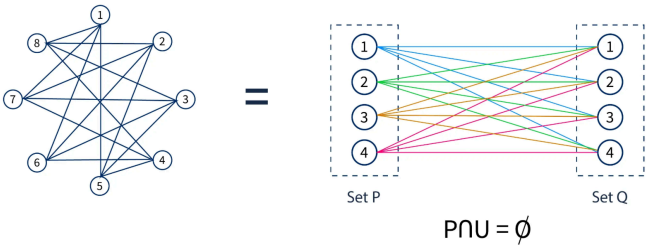
Special graph shapes

- Complete graph: every vertex connects to every other vertex
- Empty graph: zero edges
- Regular graph: every vertex has the exact same degree r (a complete graph is regular with degree n-1)
- Cycle graph: a closed loop through all n vertices
- Path graph: a straight line through all n vertices
- Complement `ḡ`: contains exactly the edges that are missing from G (everywhere G has no edge, $\bar{G}$ has one, and vice versa)

Modifying graphs

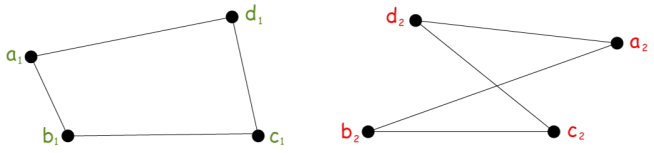
- Vertex deletion `G - v`: removes a vertex and every edge touching it
- Edge deletion `G - e`: removes only that one edge, leaves both endpoints in place
- Subgraph vs induced subgraph: a subgraph is any subset of vertices + any subset of the edges between them you choose to keep. An induced subgraph on a vertex subset keeps all the original edges that exist between the vertices you kept, you don't get to cherry-pick which edges survive

Bipartite graphs



- G is bipartite if you can split all vertices into two groups X and Y such that every single edge between the groups (never within a group)
- Key theorem: a graph (with ≥ 2 vertices) is bipartite if and only if it has no odd-length cycles

Isomorphic graphs



- Two graphs G and H are isomorphic (basically 'the same graph wearing different labels') if there's a one-to-one relabeling function f between their vertices such that edges map to edges and non-edges map to non-edges
  - If you relabel/redraw one graph and get the other exactly, they're isomorphic, the drawing and labels can look totally different, but the underlying connection structure is identical

How far apart?

- Distance `d(u, v)` = length of the shortest path between u and v. Satisfies: always ≥ 0 (and = 0 only if u = v), symmetric (d(u,v)=d(v,u)), and the triangle inequality (d(u,v) ≤ d(u,w) + d(w,v), going via a third point is never shorter
- Eccentricity `ecc(v)` = the distance from v to the farthest vertex from it
- Radius `rad(G)` = the smallest eccentricity in the whole graph (the most 'central' possible starting point's worst-case trip)

- Diameter `diam(G)` = the largest eccentricity in the graph (the worst-case trip between any two vertices)
- Center = vertices achieving the radius
- Periphery = vertices achieving the diameter
- Theorem: `rad(G) ≤ diam(G) ≤ 2·rad(G)`
  - Left half is trivial, the smallest eccentricity can't beat the largest
  - Right half means u take u, v achieving the diameter, and let c be a center vertex. By the triangle inequality, `d(u,v) ≤ d(u,c) + d(c,v)`. Each of those two pieces is at most `ecc(c) = rad(G)` (since c's eccentricity bounds its distance to anything), So `diam(G) ≤ rad(G) + rad(G) = 2·rad(G)`

28th of September (WG)

Graph Representation

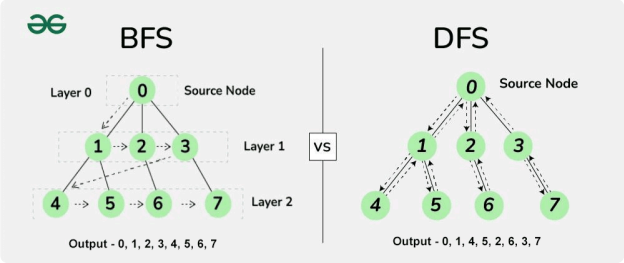
Adjacency list

- A linked list per vertex, each vertex has a list of its neighbors
  - Downsides: checking 'is there an edge between `u` and `v`?' is slow (you have to scan a list, `~|E|/|V|` steps on average), finding all edges pointing into a vertex in a directed graph is slow (have to scan everything, O(|E|))

Adjacency Matrix

- an `|V|×|V|` grid of 0s and 1s, `1` at row `i`, column `j` means there's an edge between vertex `i` and `j`
  - Downside: wastes space when the graph has few edges (lots of 0s stored for nothing)
  - Upside: checking 'is u adjacent to v' is instant (O(1) lookup)

5th of October



Breadth-First Search (BFS)

Starting from a source `s`, explores the graph ring by ring (distance 0, then 1, then 2, etc.), it uses a queue (FIFO)

Colors

- White = undiscovered
- Blue/grey = discovered, waiting in queue
- Yellow = currently being processed
- Green = fully explored (all neighbors checked)

Pseudocode

`BFS(G, s):`

```
for each u: u.color = WHITE, u.d = ∞, u.π = NIL
s.color = BLUE, s.d = 0, enqueue(s)
while queue not empty:
    u = dequeue()
    for each neighbor v of u:
        if v.color == WHITE:
            v.color = BLUE, v.d = u.d + 1, v.π = u, enqueue(v)
    u.color = GREEN
```

Running time: O(V + E), every vertex is dequeued once, every edge is looked at once (from both endpoints' adjacency lists)

What you get through BFS

- Reachability: was `t` ever discovered?
- Connectivity: everything discovered = `s`'s connected component
- Shortest paths (unweighted): `v.d` = exact number of edges on the shortest path from `s` to `v`. The `π` pointers form a BFS tree with one shortest path to every vertex

Why this is true

- Lemma 1 (One edge adds at most one to the distance): if edge `(u, v)` exists, then `δ(s,v) ≤ δ(s,u) + 1` (you can't need more than one extra step to reach `v` via `u`). Proven by: if `s` can't reach `u`, trivially true (∞ on the right). If it can reach `u`, take shortest path to `u`, add edge to `v`, gives a path length `δ(s,u)+1`, so the true shortest path to `v` can't be longer.
- Lemma 2 (BFS never underestimates): at every point in time, `v.d ≥ δ(s,v)`. Proved by induction on enqueue order using Lemma 1, each time a vertex is given a `.d` value, it's `parent.d + 1`, and by induction `parent.d ≥ δ(s,parent)`, so `v.d ≥ δ(s,parent)+1 ≥ δ(s,v)` (by Lemma 1)
- Correctness proof: combine both lemmas as a loop invariant "`v.d ≥ δ(s,v)` always, and once BFS finishes, `v.d ≤ δ(s,v)` too" (because the queue processes vertices in increasing distance order, so nothing can get a .d bigger than its real distance by the time BFS ends). Together: `v.d = δ(s,v)` exactly, for every vertex.
- This only works for edge-count distance
  - If edges have weights, BFS can give the wrong answer. Ex. `s → t` direct edge costs 10, but `s → a → t` costs 1 + 1 = 2, BFS picks `s → t` (1 edge) even though the 2-edge path is cheaper
    - this is the case unless all the edges have the same weight

BFS works the same way on directed/undirected graphs, on disconnected graphs it only reaches its own component (others stay white forever), self-loops and multi-edges don't break anything (a self-loop on an already blue/green vertex is just skipped)

Depth-First Search (DFS)

Goes as deep as possible down one branch, then backtracks. Uses a stack (LIFO) instead of a queue, that's the only structural difference from BFS

Pseudocode (non-recursive)

```
DFS(G, s):
    S = empty stack; push(S, s)
    explored = ∅
    while S ≠ ∅:
```

```
    v = pop(S)
    if v ∉ explored:
        add v to explored
        for each w in G.Adj[v]: push(S, w)
```

*A vertex can get pushed multiple times, you just skip it if it's already explored when you pop it.

Recursive

```
DFS(G):
    for each u: u.color = WHITE, u.π = NIL
    for each u: if u.color == WHITE: DFS-VISIT(G, u)
DFS-VISIT(G, u):
    u.color = BLUE
    for each v in G.Adj[u]:
        if v.color == WHITE: v.π = u; DFS-VISIT(G, v)
    u.color = GREEN
```

Running time: O(V + E) (for both cases)

Timestamps

- Each vertex gets `u.d` (discovery time) and `u.f` (finish time), basically a counter that ticks up each time you discover or finish a vertex

The Parenthesis Theorem

- For any two vertices `u, v`, their intervals `[u.d, u.f]` and `[v.d, v.f]` are either:
  - completely disjoint (neither is inside the other)
  - one is entirely nested inside the other, and the nested on is a descendant
- It's never a partial overlap like `( [ ) ]`, if `v` is discovered while `u` is still 'open' (not yet finished), `v` must finish before `u` does, because `u`'s own DFS call is what's exploring everything reachable from it, `v`'s call started inside `u`'s call, so it must return first (that's just how recursive calls work). If `v` is discovered only after `u` already finished, their whole time windows don't overlap at all
- Corollary: `v` is a proper descendant of `u` if and only if `u.d < v.d < v.f < u.f`. This means after one DFS run, 'is x an ancestor of y?' becomes a constant-time check, just compare 4 numbers
- White-path theorem: `v` becomes a descendant of `u` exactly when, at the moment `u` is discovered, there's a path from `u` to `v` made entirely of still-white (undiscovered) vertices. If the whole path is undiscovered when you start exploring `u`, DFS will walk all the way down it

---

Main Differences

	BFS tree	**DFS tree**
container	queue	stack/recursion
explores	ring by ring	branch by branch
shape	wide, shallow	long, narrow
shortest paths	yes (unweighted)	no
time	O(V + E)	O(V + E)
signature use	fewest hops	detecting cycles

5th of October (WG)

Topological Sort

It's for ordering tasks with dependencies. If edge (u, v) exists, it means '`u` must come before `v`'. Ex. undershorts → pants → shoes, socks → shoes, shirt → tie → jacket, etc. A valid topological order just needs every prerequisite to appear before the thing it unlocks, multiple valid orders can exist

- The graph must be a DAG (Directed Acyclic Graph), if there's a cycle, no valid ordering exists
- Cycle check: while doing DFS, if you ever follow an edge to a grey (currently-on-the-stack, not-yet-finished) vertex, you've found a cycle.

Algorithm:

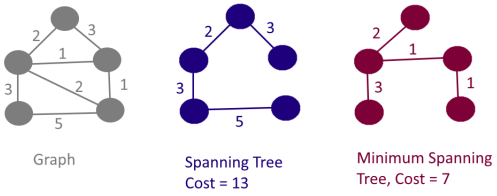
- Run DFS
- Every time a vertex turns black/green (fully finished), prepend it to the front of a results list
- The final list is the topological order

Prepending: when you finish a node `u` (color it black), every node reachable from `u` is already black (DFS fully explored them before `u` could finish), so they're already further along in the list. Putting `u` at the very front guarantees it comes before everything that depends on it in the sense that dependents get discovered via `u` and must still be ahead in finish order

- finish time order, reversed, is a valid topological order

12th of October

Minimum Spanning Trees



A spanning tree is a subset of edges that touches every vertex, with no cycle. For `n` vertices it always has exactly `n-1` edges.

Given a connected, undirected graph with edge costs, find the spanning tree with the smallest total cost is the MST problem. It shows up in network design (wiring houses as cheaply as possible), clustering, etc.

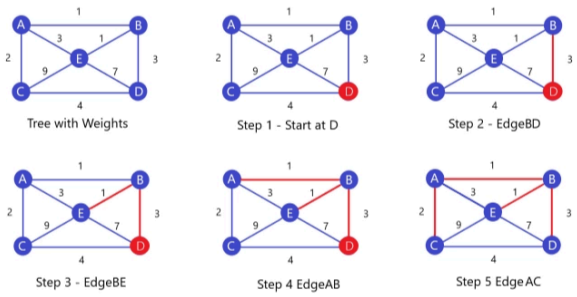
The cut property

A cut splits the vertices into two-non empty groups, a crossing edge has one end on each side. The cheapest crossing edge of any cut always belongs to some MST.

Proof by contradiction

Suppose an MST `T*` does not contain `e`, the cheapest crossing edge. Add `e` to `T*`, a tree plus one edge creates exactly one cycle, and since `e` crosses the cut, that cycle must cross the cut a second time too (to get back to where it started), using some other edge `e'`. Swap: remove `e'`, keep `e`. This new tree `T'` is still connected (removing one edge of a cycle never disconnects anything) and still has `n-1` edges, so it's still a spanning tree, but since `e` was the cheapest crossing edge, `cost(e) < cost (e')`, so `T'` is cheaper then `T*`. Contradiction: `T*` wasn't minimum after all, so no MST can skip the cheapest crossing edge

Prim's algorithm

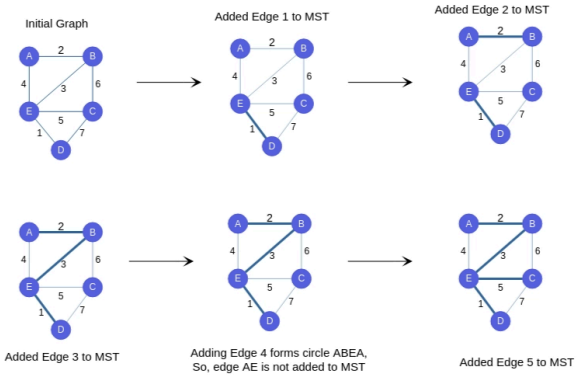


Grow one connected blob `X`, starting from any vertex. Repeatedly look at all edges leaving `X` (crossing edges), take the cheapest one, and pull its new vertex into `X`. Stop when `X` = all vertices

Each pick is safe because of the cut property (`X` vs. everything else is a cut)

- Naive running time: scanning all crossing edges every round costs O(m*n)
- With a min-heap holding the candidate edges: O(m log n)

Kruskal's algorithm



It looks at all edges, not just the crossing ones. Sort all edges by cost, cheapest first. Go through them in order: add an edge if its two endpoints are currently in different connected pieces (accept), skip it if they're already connected (it would create a cycle). Stop one you have `n-1` edges

- Each accepted edge is also justified by the cut property (the edge is the cheapest crossing edge between 'its endpoint's current piece' and 'everything else')

Union-Find

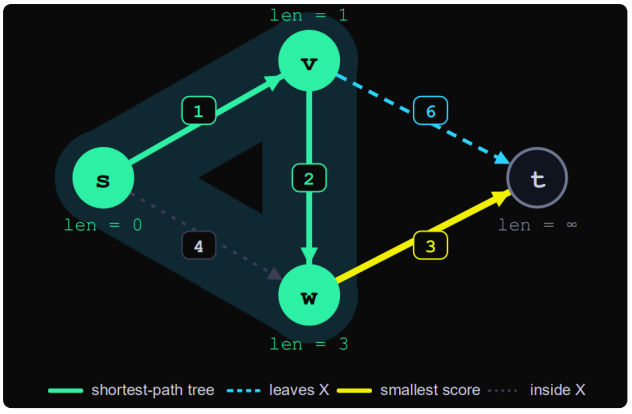
Each group has a leader, one vertex whose name represents the whole group. Every vertex stores its leader's name. `FIND(v)` just returns v's leader, two vertices are in the same group if and only if `FIND` gives the same answer. `UNION` merges two groups by relabeling every vertex of the smaller group with the bigger group's leader

Running Time

Sorting the edges costs O(m log m) = O(m log n) (since a simple graph has at most n^2 edges, log m ≤ 2 log n, and constants vanish in Big-O). The UNION calls only cost O(n log n) since UNION always relabels the smaller side, every time a single vertex `v` gets relabeled, the group it now belongs to is at least double the size of its old group. A group can't double in size more than $\log_2 n$ times before it covers everyone, so each vertex gets relabeled at most $\log_2 n$ times, ever, across the whole algorithm. Overall Kruskal is O(m log n), same as Prim

---

Dijkstra's algorithm (shortest paths)



Given a directed graph with non-negative edge lengths and a source `s`, find the shortest-path distance from `s` to every vertex

*Can't use BFS, because that counts edges, not length. It only works when every edge has weight 1

It has the same greedy shape as Prim, but with a different score. Grow `X` from `s`. Each vertex `v` already in `X` has `len(v)` = its true shortest distance. For every edge `(v, w)` leaving `X`, compute `score(v, w) = len(v) + length(v, w)`. Pull in whichever outside vertex has the smallest score, and that score becomes its final `len`

Pseudocode

```
X := {s}
while an edge leaves X:
    pick edge (v,w) with smallest score = len(v) + length(v,w)
    add w to X
    len(w) := score
```

Proof by induction

Base case: `X = {s}`, `len(s) = 0 = dist(s,s)` (no path can be negative since all lengths ≥ 0)

Inductive hypothesis: assume for every vertex already in `X`, `len =` the true shortest distance

Inductive step: Dijkstra picks edge `(v*, w*)` with the smallest score and adds `w*`. We need `len(w*) = dist(s, w*)`

- One direction is easy: `len(w*) = len(v*) + length(v*,w*)` is literally a real path (shortest path to `v*`, by hypothesis, plus one edge), so `dist(s,w*) ≤ len(w*)`, you can't do worse than a path you can point to
- The other direction needs more care: could there be some *other*, cheaper route to `w*`? Take any path `P` from `s` to `w*`. Since `s ∈ X` and `w* ∉ X`, `P` must cross out of `X` at some point — split it into: the piece inside `X` up to some vertex `y`, the one edge `(y,z)` that leaves `X`, and the rest of the path to `w*`. Lower-bound the whole path's length: it's at least `dist(s,y) + length(y,z) + 0` (dropping the remaining non-negative piece) `= len(y) + length(y,z)` (by the hypothesis, since `y ∈ X`) `≥ len(w*)` (because Dijkstra chose the edge with the *smallest* score overall, and `(y,z)` is some crossing edge — its score can't beat the one Dijkstra picked)
- So every possible path to `w*` costs at least `len(w*)`, meaning `dist(s,w*) ≥ len(w*)`. Combined with the other direction: `dist(s,w*) = len(w*)` exactly

Running time

Same as Prim, repeatedly asking for a minimum, so a min-heap gives O(m log n)

12th of October (WG)

A*

If your graph is huge and you only want the shortest path to one specific target, Dijkstra still explores in every direction equally, wasting time on routes that obviously aren't going toward your goal

Use a heuristic `h(a, b)`, an estimate of the distance from `a` to the goal `b`. A* expands the vertex minimizing `path(start, a) + h(a, goal)` instead of just `path(start, a)`. This steers the search toward the goal instead of outward in all directions

Admissibility

- The heuristic must be admissible: `h(a, b) ≤ true shortest distance (a, b)`, it must never overestimate. You want it as high as possible while staying admissible (straight-line distance is a classic admissible heuristic for map routing, you can never beat flying in a straight line)
- If the heuristic is too low (far from the true distance): A* still finds the shortest path, but expands more nodes than necessary, closer to plain Dijkstra, less efficient
- If the heuristic is not admissible (overestimates somewhere): A* can return the wrong (non-shortest) answer

You can't stop the moment you first reach the goal in your search, a cheaper route might still be sitting in the priority queue. You only stop once the goal is popped off the top of the queue (it's current best candidate), because another path could still prove shorter until then